

Project Proposal - Edge-Based Vision System for Manufacturing Process Adherence

Erin Walter | CIS 589 WI 2026

Abstract — This paper proposes an edge-based computer vision system to monitor operator process adherence in automotive manufacturing. The system uses real-time pose estimation to track which bins an operator picks from and in what order, comparing the observed sequence against a trim-level-specific expected sequence delivered via MQTT. When a deviation is detected, the operator receives an immediate on-screen alert. All inference runs locally on a Jetson Nano to ensure low latency and preserve operator privacy. Session logs are forwarded to a PC over MQTT for long-term quality analysis. The system is evaluated on alert latency and network bandwidth efficiency compared to a video-streaming baseline.

I. PROBLEM STATEMENT

On the manufacturing line, plant managers, process coaches, and industrial engineers are interested in having a system to help operators with completing the correct steps in the correct order while assembling vehicles. All trims of the vehicle are assembled on the same line, so it is important that an operator pick the correct part and install the correct part for the given trim. In the normal process, when a vehicle comes through the line, the operator will see a trim configuration, and must then read the process for that configuration and follow the steps, including picking the right part from the bin and installing it correctly. If this process was monitored and feedback was given to the operators if they were installing an incorrect part or not following the process correctly, this could help with vehicle quality and with plant efficiency, which will save money on overtime as well as minimize recalls later on down the line.

II. SYSTEM DESIGN

This project proposes an edge-based computer vision system that monitors operator actions in real time and verifies they are following the correct step sequence for the current vehicle configuration. When the cycle starts, the system will receive the trim level via MQTT, and then use a webcam to detect which bins the operator picks from and in what order. If a deviation is detected, the operator is immediately alerted with a visual warning.

All real-time processing runs locally on a Jetson Nano, while error logs and session data are sent to a PC for quality tracking and long-term analysis. This ensures that alerts to the operator have low latency and the operator identity is kept private since videos are not being saved. Additionally, since plant floors have limited bandwidth, sending only JSON data and not entire video clips or frames through the network helps keep bandwidth low on the plant floor.

System Components

For this system, the following components will be utilized: Jetson Nano, webcam attached to the Jetson Nano, a screen for alerting attached to Jetson Nano, Mosquitto MQTT broker, a PC, a python application on edge for running AI models, and a python application on PC for running analysis

Data Sources

The following data sources will collect data during this project: the webcam video feed, MQTT messages, and a pre-configured config lookup table which will map trim level to the expected sequence of events

Data Processing Pipelines

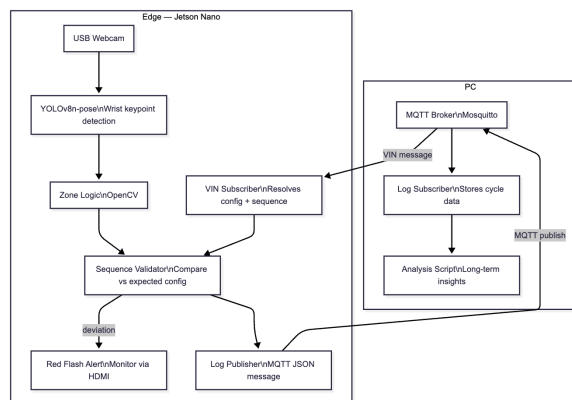
- MQTT Message received -> trim extracted
-> expected step sequence loaded

- Webcam feed captured -> YOLO detects operators wrist keypoint coordinates
- OpenCV checks which zone the wrist coordinate is in -> keep track of step sequence
- Zone entry sequence compared against expected sequence for active config
- If a deviation is found -> flash alert on operator's screen to alert them
- Session log such as steps completed, errors, trim level, cycle time, all sent to laptop for further processing

Component Communication

- MQTT message with trim published to broker -> Jetson Nano subscribes to this and receives expected trim
- Webcam -> Jetson Nano connected via USB, video frames captured and analyzed locally
- Jetson Nano -> Monitor connected via display port wire
- Jetson Nano -> Laptop connected via MQTT, nano publishes message with cycle details and laptop is subscribed.

Architectural Diagram



III. IMPLEMENTATION PLAN

During this semester I plan to accomplish the following parts of the system:

- Set up Mosquitto MQTT Broker on my laptop and create topics for the

configuration cycle messages and the cycle data message.

- Write the logic for tracking wrists utilizing YOLOv8n-pose model.
- Write the logic on edge for creating zones in the video frame using OpenCV (will be a manual, simple process for the scope of this class)
- Write the logic on edge for tracking an operator's sequence based on the zones
- Define three sequences, A, B, and C that can be used for testing and validating the sequence order logic
- Write the logic to flash an alert on screen if the sequence is not followed or an incorrect part is picked for the specific trim level.
- Write the logic to publish cycle data from the edge device to the MQTT broker once a cycle is completed
- Write the logic to receive the data from the MQTT broker on the laptop and save the data for future analysis
- Write some simple logic to analyze the data to show how "long-term insights" can be gleaned from the data collected on edge.

Tools Needed

- Hardware: Jetson Nano, PC, USB Webcam, Computer Monitors
- Software/Libraries: Mosquitto MQTT Broker, YOLOv8n-pose, OpenCV
- Languages: Python for all development

IV. EVALUATION PLAN

Since real-time alerting on the line is one primary benefit of the system, I will measure the latency between performing a step incorrectly and the alert showing up on the screen to tell me that I picked the wrong part. Additionally, since network bandwidth is important to keep low, I will compare the amount of bandwidth used to send a MQTT JSON message from the Jetson to the PC versus a simulated test of sending video frames from the Jetson to PC by creating an HTTP endpoint and measuring that bandwidth.